

用户管理系统

安全漏洞分析与修复实验报告

安全实验教学项目

安全

实验日期：年 月 日

实验目的：识别并修复 登录模块的四类安全漏洞（密码明文、会话伪造、信息泄露、暴力破解）

部署地址：

目 录

一、实验概述

实验环境

实验目的

项目架构

二、漏洞一（高危）：用户密码明文存储与前端回显

漏洞原理

漏洞复现操作（ 实验流程 ）

实际危害

修复方案与代码对比

长期加固建议

三、漏洞二（高危）：会话密钥硬编码

漏洞原理

漏洞复现操作（ 伪造实验流程 ）

实际危害

修复方案与代码对比

长期加固建议

四、漏洞三（中危）：注释泄露管理员凭证

漏洞原理

漏洞复现操作

实际危害

修复方案与代码对比

长期加固建议

五、漏洞四（中危）：登录接口无防暴力破解机制

漏洞原理

漏洞复现操作（ 字典爆破流程 ）

实际危害

修复方案（ 代码实现 ）

长期加固建议

六、修改对照总表

七、修复验证测试

测试用例与结果

抓包验证说明

八、实验总结与安全开发规范

实验总结

安全开发规范

延伸加固方案

一、实验概述

实验环境

开发语言：

框架：（ ）

密码工具：（

,)

安全测试工具：

字典爆破工具：

（)

分析工具： 库

网络抓包工具：（监听)

部署地址：

实验日期：年 月 日

实验目的

理解 应用中密码明文存储与传输的安全风险

掌握 签名机制及密钥泄露导致的会话伪造攻击

认识前端注释泄露敏感信息的危害

掌握 抓包、字典爆破等渗透测试基本操作

学习使用 进行密码哈希存储与安全比对

学习登录接口防暴力破解的基本策略

项目架构

```
flask-app-vuln/  
  app.py                # main app (routes, user DB, login logic)  
  templates/  
    base.html           # base template (navbar, layout)  
    index.html          # home page (user info display)  
    login.html          # login page  
  static/css/  
    style.css           # global styles
```

二、漏洞一（高危）：用户密码明文存储与前端回显

漏洞原理

在 的 字典中，密码以纯文本形式直接存储（例如 ）。登录验证时，使用 运算符直接将用户输入的密码与字典中的明文密码进行字符串比较。登录成功后，将包含字段的完整用户字典传递给模板，模板中的 将密码原文输出到 页面。

该漏洞违反 中（加密机制失效）安全要求。正确的做法是使用不可逆的哈希算法（如 、 、 ）对密码进行摘要存储，且永远不应将密码字段传递给前端模板渲染。

漏洞复现操作（实验流程）

- 步骤一：启动 应用，访问
- 步骤二：在浏览器中使用默认管理员账号 登录
- 步骤三：登录成功后跳转到首页，页面直接显示明文密码
- 步骤四：按 打开开发者工具 面板，定位到用户信息列表，可直接看到密码原文
- 步骤五：启动 ，配置浏览器代理（ ）
- 步骤六：使用 拦截登录 请求：

步骤七：观察请求体，密码以明文形式在 中传输，无任何加密保护

步骤八：将拦截到的数据包发送至 ，可重复发送验证登录行为

实际危害

- 数据库泄露（如字典被 ）后，所有用户密码直接暴露
- 攻击者可利用泄露的密码在其他平台尝试撞库攻击（ ）
- 前端页面暴露密码，任何能查看页面源代码的人都能获取
- 明文传输，中间人攻击（ ）可截获登录凭证

修复方案与代码对比

修复一： 新增安全库导入与密码哈希化

修改项	修改前	修改后
密码存储		
密码比对		
导入语句	无安全导入	

修复二： 禁止模板输出密码字段（注意：最优方案是后端完全不传递字段，而非仅前端掩码）

修改项	修改前	修改后
密码显示	直接输出密码原文	(已加密保护) 或彻底移除字段渲染

```
<!-- BEFORE (vulnerable line) -->
<li><strong></strong>{{ user.password }}</li>

<!-- AFTER (fixed) -->
<li><strong></strong>*****</li>
<!-- BEST: dont render password field at all -->
<!-- exclude password from dict passed to template -->
```

长期加固建议

长期加固建议

- 使用 或 替代 作为密码哈希算法
- 后端模板渲染时彻底不传递 字段，而非仅前端掩码
- 线上部署强制启用 ，避免 传输密码明文被抓包
- 禁止在任何日志中打印用户密码
- 定期使用密码泄露检测工具（如 ）检查密码是否已泄露

三、漏洞二（高危）：会话密钥硬编码

漏洞原理

使用基于 库的签名机制来保护 。服务器使用 对 数据进行 签名，以确保数据在传输过程中未被篡改。

当 设置为固定的弱密钥（如 ）时，攻击者可以从源代码中获取该密钥，利用 或 库对劫持到的 进行解签、篡改载荷后重新签名，从而伪造任意用户的会话身份。

注意：此漏洞属于 会话管理不安全（ ），严格来说不是 弱密码 问题。正确术语为： 签名密钥硬编码导致的会话伪造攻击（ ）。

漏洞复现操作（伪造实验流程）

步骤一：正常登录系统（ ），使用 或浏览器开发者工具获取

步骤二：提取 值（默认的 名为 ）

步骤三：在终端中使用 工具解签 数据：

```
# decode session cookie
flask-unsign --decode --cookie '<session_value>'

# if secret key is known, verify by unsigning
flask-unsign --unsign --cookie '<session_value>' --secret 'dev-key-2025'
```

步骤四：篡改 载荷，将 改为：

```
# forge session for any user
flask-unsign --sign --cookie '{"username": "admin"}' --secret 'dev-key-2025'
```

步骤五：将伪造的 注入浏览器，刷新页面后可无需密码以 身份登录

步骤六：使用 修改请求中的 头，验证无需密码即可登录

实际危害

- 攻击者可伪造任意用户的 ，实现无需密码的身份冒充
- 可伪造管理员会话，获取系统最高权限
- 会话劫持配合信息泄露可导致完全的系统沦陷
- 由于密钥硬编码在代码中，所有部署实例使用相同密钥，漏洞影响面广

修复方案与代码对比

修复说明一：禁止使用 作为修复方案，因为每次重启服务后所有已有 会失效，强制全部用户下线是生产环境不可接受的可用性問題。

修复说明二：正确的方案是从环境变量中读取 。开发环境可在 文件中配置固定密钥（但需打印醒目警告提醒不应用于生产），生产环境必须从环境变量或密钥管理服务（如 ）中读取。

```
# CORRECT approach (production)
import os
import secrets
```

```
app.secret_key = os.environ.get('SECRET_KEY')
if not app.secret_key:
    if os.environ.get('FLASK_ENV') == 'development':
        # dev only: allow default with prominent warning
        print("WARNING: Using default SECRET_KEY in development!")
        app.secret_key = 'dev-secret-key-do-not-use-in-production'
    else:
        raise ValueError("SECRET_KEY environment variable is required")
```

长期加固建议

生产环境 通过环境变量注入，长度至少 位随机字符

限制 有效期，设置

增加 绑定校验（ ）

敏感操作（修改密码、转账等）要求二次身份验证

定期轮换，配合 过期策略

四、漏洞三（中危）：注释泄露管理员凭证

漏洞原理

文件第 行包含一条 注释：

```
<!-- DEBUG - default admin account: admin / admin123 -->
```

注释虽然在浏览器页面上不可见，但任何用户通过 查看页面源代码（
开发者工具）即可直接读取注释内容，获取管理员的登录凭证。

这是典型的 前端敏感信息泄露（
），属于
中 的范畴。

漏洞复现操作

步骤一：访问

步骤二：在页面上右键 选择 查看页面源代码（或按）

步骤三：在 顶部看到注释中的账号密码：

步骤四：使用 拦截登录页面响应，中同样包含该注释

步骤五：直接使用泄露的凭证登录系统，验证信息真实性

实际危害

低权限用户可获取管理员账号密码，实现越权访问

源码管理（）中若包含该文件，历史版本将永久记录该凭证

搜索引擎可能索引页面内容，导致凭证泄露范围扩大

修复方案与代码对比

修改项	修改前	修改后
注释内容		请联系管理员获取账号

```
<!-- BEFORE (vulnerable) -->
<!-- DEBUG - default admin account: admin / admin123 -->
```

```
<!-- AFTER (fixed) -->
<!-- DEBUG - contact admin for credentials -->
```

长期加固建议

建立上线前检查清单（），全面清理所有调试注释

禁止在前端、、中写入任何密钥、接口凭证

使用环境变量控制调试模式，生产环境禁用 信息输出

在 流程中加入关键词扫描（如、、）

默认账号密码应在系统初始化时强制修改，不得写死在代码或页面中

五、漏洞四（中危）：登录接口无防暴力破解机制

漏洞原理

当前登录接口 没有任何频率限制、失败次数锁定或验证码机制。攻击者可以使用加载常见密码字典，对已知用户名（如 ）进行批量密码枚举，通过响应内容的差异（用户名或密码错误！跳转到首页）判断密码是否匹配。

这是典型的 不安全的登录接口设计，属于 中 的范畴。

漏洞复现操作（字典爆破流程）

步骤一：启动 ，配置浏览器代理后访问

步骤二：在登录表单中输入 任意密码，点击登录，拦截到 请求：

```
POST /login HTTP/1.1
Host: 127.0.0.1:5000
...
username=admin&password=test123
```

步骤三：右键数据包

步骤四：在 的 选项卡中，清除所有默认 位置

步骤五：选中 中的，点击 标记为 位置

步骤六：切换到 选项卡，点击 加载下载的字典文件（如 或 ）

步骤七：启动 ，观察每个请求的响应长度和状态码

步骤八：当响应长度明显不同（登录成功跳转首页 返回错误提示）时，对应的密码即为正确密码

步骤九：找到 的正确密码，验证爆破成功

实际危害

无锁定机制，攻击者可无限枚举密码

字典爆破工具（ 、 、 等）可在数分钟内完成对弱密码的破解

结合漏洞三的泄露信息，爆破范围可更精准

大量错误登录请求可能影响服务器性能

修复方案（代码实现）

以下提供两种方案的代码实现。方案一为本地开发用的内存计数器（简单但重启后失效）；方案二为生产环境推荐的持久化计数器（支持多进程多服务器）。

方案一：内存计数器（适合本地开发单进程部署）

```
# Option 1: in-memory counter (dev only)
from collections import defaultdict
from datetime import datetime, timedelta

login_attempts = defaultdict(lambda: {'count': 0, 'lock_time': None})
MAX_ATTEMPTS = 5
LOCK_DURATION = timedelta(minutes=5)
```

```

@app.route("/login", methods=["GET", "POST"])
def login():
    error = None
    if request.method == "POST":
        username = request.form.get("username", "")
        password = request.form.get("password", "")
        client_ip = request.remote_addr

        # check if locked
        record = login_attempts[client_ip]
        if record['lock_time'] and datetime.now() < record['lock_time']:
            remaining = (record['lock_time'] - datetime.now()).seconds
            error = f" {remaining} "
            return render_template("login.html", error=error)

        if username in USERS and check_password_hash(USERS[username]["password"], password):
            # login OK, clear failure record
            login_attempts[client_ip] = {'count': 0, 'lock_time': None}
            session["username"] = username
            return render_template("index.html", user=USERS[username])
        else:
            record['count'] += 1
            if record['count'] >= MAX_ATTEMPTS:
                record['lock_time'] = datetime.now() + LOCK_DURATION
            error = ""
    return render_template("login.html", error=error)

```

方案二：持久化计数器（推荐生产环境使用，服务重启不丢失、支持多进程多服务器）

```

# Option 2: Redis persistent storage (production)
import redis
from datetime import timedelta

r = redis.Redis(host='localhost', port=6379, db=0, decode_responses=True)

def check_login_lock(username):
    """ (, ) """
    key = f"login_failures:{username}"
    failures = r.get(key)
    if failures and int(failures) >= 5:
        ttl = r.ttl(key)
        if ttl > 0:
            return True, f" {ttl} "
    return False, None

def record_login_failure(username):
    """ 15 """
    key = f"login_failures:{username}"
    r.incr(key)
    r.expire(key, 900) # auto-expire after 15 minutes

def clear_login_record(username):
    """ """

```

```
r.delete(f"login_failures:{username}")
```

长期加固建议

- 登录增加图形验证码（ ）或人机验证（ ）
- 使用 等持久化存储记录登录失败次数（避免服务重启后计数重置）
- 限制单 单位时间内的请求频率（ ，如使用 ）
- 增加登录通知机制：异地登录、新设备登录发送告警
- 强制要求用户设置符合复杂度要求的强密码

六、修改对照总表与 映射

修改对照总表

漏洞编号	涉及文件	修改位置	修改前	修改后	风险等级
密码明文		第行导入语句	无安全库导入 导入		高危
密码明文		第行字典 哈希字符串			高危
密码明文		第行登录比对	直接字符串比对 安全哈希比对		高危
密码明文		第行信息传递	明文密码原文到 掩码或彻底不传字段		中危
密钥		第行导入模块	未导入		高危
密钥		第行	固定弱密钥环境变量读取		高危
注释泄露		第行注释	完整凭证泄露 请联系管理员获取账号不含任何敏感信		中危
暴力破解		函数认证逻辑	无限制可无限次尝试登录 失败锁定分钟持久化计数		中危

映射

以下将四类漏洞映射到 标准分类，体现报告的专业性和行业参考价值：

漏洞编号	漏洞名称	映射	分类说明	参考
	密码明文存储与回显		加密机制失效密码应哈希存储	高危
	密钥硬编码		不安全的设计会话管理缺陷	高危
	注释泄露管理员凭证		安全配置错误前端信息泄露	中危

	登录接口无防暴力破解		身份认证失败缺乏速率限制	中危
--	------------	--	--------------	----

测试编号	测试场景	预期结果	验证结果
	正确密码登录	登录成功跳转首页	通过
	错误密码登录	显示用户名或密码错误！	通过
	首页查看密码已登录状态	密码显示为	通过
	查看 页面源代码	无账号密码注释仅显示请联系管理员	通过
	伪造	密钥硬编码已移除无法伪造	通过
	抓登录请求查看密码传输 存储层：已修复哈希传输层：需待办		存储层 通过传输层 待办
	连续错误密码次以上	第次起提示锁定信息	通过
	锁定后等待分钟再次登录	锁定解除可正常登录	通过

```
# test_security_fixes.py
import pytest

def test_password_not_returned_in_response(client):
    """
    client.post('/login', data={'username': 'admin', 'password': 'admin123'})
    response = client.get('/')
    assert 'admin123' not in response.text
    assert '*****' in response.text
    """

def test_brute_force_lockout(client):
    """5"""
    for i in range(6):
        resp = client.post('/login', data={'username': 'admin', 'password': 'wrong'})
        if i >= 5:
            assert '' in resp.text or ' ' in resp.text

def test_html_comment_no_password(client):
    """ admin/admin123"""
    response = client.get('/login')
    assert 'admin123' not in response.text

def test_logout_clears_session(client):
    """ session """
    client.post('/login', data={'username': 'admin', 'password': 'admin123'})
    client.get('/logout')
    response = client.get('/')
    assert '' in response.text
```

八、实验总结与安全开发规范

实验总结

通过本次实验，我们从 安全攻防的角度完成了一次完整的漏洞发现、复现与修复演练。

实验成果总结如下：

发现了 个安全漏洞：密码明文存储（高危）、 密钥硬编码（高危）、
注释泄露凭证（中危）、登录接口无防暴力破解（中危）
使用 完成了抓包拦截、字典爆破等渗透测试操作
使用 实现了密码的 哈希存储与安全比对
使用 随机生成 密钥，消除了会话伪造风险
清理了前端注释中的敏感信息，消除了信息泄露途径
实现了基于 的登录失败计数与锁定机制，防御暴力破解

特别需要注意的是，安全是一个系统工程，仅修复单个漏洞是不够的。密码哈希解决了存储安全问题，但传输层仍需要保护；密钥随机化解决了会话伪造，但持久化与过期策略仍需规划；防爆破机制虽能抵御字典攻击，但配合验证码和限流效果更佳。

安全开发规范

以下为本次实验总结的 安全开发规范，可作为团队开发的标准检查清单：

密码存储使用	登录次失败即锁定分钟
密码比对用	全站启用（ ）
通过环境变量注入	日志中禁止记录密码和
前端注释禁止写任何凭证	生产环境关闭 模式

延伸加固方案

以下为超出本次实验范围，但强烈建议实施的额外安全加固措施：

传输层安全（ ）

使用 免费证书 反向代理，强制全站
设置（ ）头，禁止降级到
配置安全加密套件，禁用 、 、 等弱加密算法

（跨站请求伪造）防护

应用应集成 扩展，为每个表单生成
所有 请求验证 有效性
设置 属性为 或

```
# CSRF protection with Flask-WTF
from flask_wtf.csrf import CSRFProtect

csrf = CSRFProtect(app)

# in login.html, add CSRF Token:
```



```
# <form method="post">
#     <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
#     ...
# </form>
```

服务端会话存储（替代客户端签名）

使用 `Flask-Session` 将数据存储在服务端

客户端只存储 `session_id`，彻底避免会话篡改风险

支持多服务器共享 `session_id` 状态

```
# server-side sessions (Flask-Session + Redis)
from flask_session import Session
import redis

app.config['SESSION_TYPE'] = 'redis'
app.config['SESSION_PERMANENT'] = False
app.config['SESSION_USE_SIGNER'] = True
app.config['SESSION_REDIS'] = redis.from_url('redis://localhost:6379')
Session(app)
```

数据验证与输出编码

使用 `Flask-WTF` 或 `Flask-Validator` 对用户输入进行严格校验

模板中启用 `autoescape` 自动转义（默认已启用），防止

避免使用 `render_template_string` 过滤器或 `render_template` 直接渲染未经过滤的用户输入

安全头配置

`X-Frame-Options: deny`：限制资源加载来源，防御 `Clickjacking` 和数据注入

`X-XSS-Protection: 1; mode=block`：防止点击劫持（`Content-Security-Policy`）

`X-Content-Type-Options: nosniff`：防止 `Content-Type` 类型混淆攻击

`Cache-Control: no-cache`：控制

信息泄露

日志与监控

记录所有登录尝试（成功失败），包括 `IP`、时间戳、

设置异常行为告警：同一账号异地登录、短时间内大量失败等

定期审计访问日志，排查异常请求模式

依赖库安全

定期使用 `Safety` 或 `OWASP Dependency-Check` 扫描依赖库的已知漏洞

更新 `requirements.txt` 及所有依赖到最新稳定版本

使用 `pip freeze --hash` 锁定版本，避免意外升级引入不兼容

报告完

安全是一个过程，不是一个产品。